

# **Blatt 6**

## Web-Security und Intrusion Prevention

Luis Staudt

## Aufgabe 1: Die Webanwendung

Es laufen zwei Flask-Anwendungen: das Gästebuch aus Blatt 5 (Port 5000, mit der dort gezeigten XSS-Lücke) und eine Online-Banking-Anwendung (Port 5001). Benutzer der Bank sind alice (alice123), bob (bob123) und eve (eve123), jeweils mit einem Kontostand. Nach dem Login setzt die Bank ein ungeschütztes Sitzungscookie `session_id` ohne SameSite- oder HttpOnly-Flag.

## Aufgabe 2: Die Banking-Seite verstehen

Eine Überweisung von Alice an Bob über 10 EUR zeigt in der Entwicklerkonsole den zugehörigen Aufruf. Die Überweisung läuft über einen einfachen GET-Request:

```
1 GET /transfer?to=<empfaenger>&amount=<betrag> (Cookie: session_id=...)
```

Benötigt werden somit nur die Parameter `to` und `amount`; die Authentifizierung erfolgt allein über das automatisch mitgesendete Sitzungscookie. Genau das macht die Anwendung anfällig für CSRF.

```
1 curl -b "session_id=..." "http://127.0.0.1:5001/transfer?to=bob&amount=10"
2 -> Kontostand: 1000 -> 990
```

### Aufgabe 3: CSRF über die XSS-Lücke im Gästebuch

Überwiesen werden sollen 500 EUR an Eve. Dazu wird ein `<img>`-Tag ins Gästebuch eingetragen, dessen `src` auf den Transfer-Endpunkt der Bank zeigt. Das Laden eines Bildes bleibt dabei unauffällig:

```
1 Schoenes Bild! 
```

Ist Alice bei der Bank eingeloggt und ruft das Gästebuch auf, versucht ihr Browser das vermeintliche Bild zu laden und sendet dabei ihr Banking-Cookie automatisch mit. Die Überweisung wird im Kontext von Alice ausgeführt:

```
1 Kontostand Alice: 1000 -> 500 (500 EUR an Eve ueberwiesen)
```

### Sec-Fetch-Header der gefälschten Anfrage

Serverseitig wurden die Header der eingehenden `/transfer`-Anfrage protokolliert:

```
1 Sec-Fetch-Site: same-site
2 Sec-Fetch-Mode: no-cors
3 Sec-Fetch-Dest: image
4 Referer:      http://127.0.0.1:5000/
5 Cookie:      session_id=...; secretcookie=itsasecret; ...
```

`Sec-Fetch-Site` trägt den Wert `same-site`, weil beide Anwendungen denselben Host `127.0.0.1` nutzen; der Port gehört nicht zur Site. Der Browser sendet das Cookie deshalb trotz des `SameSite`-Standards (`Lax`) mit. Bei verschiedenen Sites würde `Lax` den Angriff bereits erschweren. `Sec-Fetch-Dest: image` weist aus, dass die Anfrage von einem `<img>`-Tag stammt, und `Referer` zeigt auf das Gästebuch.

### Verhalten bei POST-basierter Überweisung

Bei Verwendung von `POST` liesse sich der Angriff nicht mehr über ein `<img>`-Tag auslösen, da Bilder ausschliesslich `GET`-Anfragen erzeugen. Möglich bliebe er dennoch: Erforderlich wäre ein automatisch abgesendetes `HTML`-Formular mit `method="post"`, das per `JavaScript` submittet wird. Über die `XSS`-Lücke lässt sich ein solches Formular ebenfalls einschleusen. `POST` allein schützt somit nicht vor `CSRF`.

## Aufgabe 4: Absicherung mit CSRF-Token

Die Bank wird um CSRF-Tokens erweitert. Beim Login wird pro Sitzung ein zufälliges Token erzeugt und serverseitig gespeichert. Das Transfer-Formular enthält das Token als verstecktes Feld, und /transfer wird auf POST umgestellt und prüft das Token:

```

1 def set_session(user):
2     sessions[session] = {"user": user, "csrf": get_random_string(32)}
3
4 @app.route("/transfer", methods=["POST"])
5 def transfer():
6     s = get_session(request.cookies.get("session_id"))
7     if not s: return redirect("/")
8     if request.form.get("csrf_token") != s["csrf"]:
9         abort(403)
10    ...

```

Zusätzlich wird das Sitzungscookie mit httponly=True und samesite="Strict" gesetzt.

Die Tests bestätigen die Wirkung:

```

1 # alter <img>-CSRF (GET) -> Methode nicht mehr erlaubt
2 GET /transfer?to=eve&amount=500 -> 405
3 # gefälschter POST ohne gültiges Token -> abgewiesen
4 POST /transfer (ohne csrf_token) -> 403
5 # legitime Ueberweisung mit Token aus dem Dashboard -> funktioniert
6 POST /transfer (csrf_token=..., to=bob, amount=50) -> 302, Kontostand 1000 -> 950

```

Da ein Angreifer das sitzungsgebundene Token nicht erraten kann, wird die gefälschte Überweisung mit 403 abgewiesen. Legitime Überweisungen aus dem Dashboard funktionieren weiterhin.